

# Relatório de Status por Sprint — repositório GitHub `diegolsarmond/WorkSafety`

## Resumo executivo

O repositório `diegolsarmond/WorkSafety` está estruturado como um **monorepo** com backend (Django + DRF + Celery), frontend (SPA), infraestrutura Docker e um módulo web/admin separado ("WorkSafetyWeb"), com foco em inspeções de segurança, upload de evidências e processamento assíncrono. <sup>1</sup>

Em termos de "sprints", **não há evidências no repositório de Issues, PRs, Milestones ou Project Boards** sendo usados como fonte primária de rastreo (as buscas retornaram vazio). Assim, o mapeamento por sprint foi feito a partir de: (a) artefatos internos do repositório que explicitam sprints e escopo (READMEs e instruções), (b) commits com "tickets" BE-xx e mensagens descritivas, e (c) a OS/ planejamento (PDF enviado por você) como referência de backlog e critérios de aceite por sprint. `filecite\turn0file0`

**Status geral (alto nível):** - **Sprint 1-2 (fluxo básico + captura/sincronização):** majoritariamente **pronto**, com pendência relevante em **reset de senha no frontend** (há página, mas a função de reset não está implementada). <sup>2</sup>

- **Sprint 3 (ciclo de vida + fila IA + relatórios):** base técnica existe (pipeline assíncrono, endpoints e UI de apoio), mas o **relatório PDF** e a **página de relatórios** mostram sinais de "MVP/stub" e inconsistências de autenticação no cliente. <sup>3</sup>

- **Sprint 4 (LGPD/anonimização + IA real + integração Olímpia):** há implementação importante de LGPD (base legal, anonimização de rostos, trilha de auditoria), mas há **lacunas técnicas**: (a) **detecção/anonimização de placas está marcada como TODO**, (b) **serialização/contrato de API de evidência LGPD está inconsistente** (serializer retorna valores "placeholder" e não expõe campos LGPD corretamente), e (c) o pipeline de IA ainda usa **cliente Mock** e um doc do repositório explicitamente diz que o cliente real depende do contrato.

---

## Fontes e metodologia de levantamento

A análise foi conduzida em três camadas:

Primeiro, foi feito o inventário do repositório e documentos internos (README raiz + READMEs de backend/frontend) para identificar convenções de sprint e fluxos do sistema.

Em seguida, foram levantadas "tarefas" via **histórico de commits**, priorizando commits que funcionam como unidades de entrega (ex.: "BE-01", "BE-05", "Implement ..."), já que não há Issues/PRs/Milestones/Boards em uso. O commit "BE-05" (LGPD/GDPR) foi aprofundado por conter mudanças amplas, inclusive documentação e logs de migração. <sup>3</sup>

Por fim, o backlog e definição de sprints do documento de OS/planejamento (PDF enviado) foi usado como **baseline** para classificar itens como “pronta” vs “pendente”, por comparação do que existe no código e nos artefatos do repositório com o que é exigido por sprint. `filecite{turn0file0}`

Limitação relevante: a documentação do portal Olímpia **não foi recuperável automaticamente** via web crawler, então o relatório fornece um **guia robusto porém “assumido”** para autenticação/headers/body (com pontos claros para validação manual no Swagger do Olímpia). O endpoint informado por você é tratado como fonte de verdade de caminho ( `/v2/segurança-por-imagem/infer` ).

## Mapeamento de sprints e inventário de tarefas

**Convenção de “Sprint” encontrada no repositório** - Backend e frontend possuem textos que descrevem “Sprint 1” e “Sprint 3”, além de guias de pipeline.

- Não há milestones/boards/labels padronizados no GitHub para “Sprint X”. Por isso, a organização do relatório por sprint foi feita por **correspondência de escopo funcional** (OS/planejamento) `filecite{turn0file0}` e por **commits de entrega** (ex.: BE-01...BE-05).

**Observação importante sobre endpoints** O backend expõe rotas sob o prefixo `/api/` (ex.: `/api/auth/`, `/api/assessments/`, `/api/admin/`).

Parte da documentação interna (por exemplo, instruções específicas) apresenta caminhos alternativos/antigos (ex.: `/upload/` em vez de `/evidences/` ), o que é um item de “pendência de alinhamento” para evitar integração quebrada e confusão em homologação.

## Tabelas de status por sprint

### Sprint 1 — Autenticação, usuários e base do app

| ID/issue/PR                    | Título                                                            | Autor         | Labels              | Status        | Arquivos alterados                                                                                                             | Resumo técnico                                                                                                          |
|--------------------------------|-------------------------------------------------------------------|---------------|---------------------|---------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Commit<br><code>acd4b0e</code> | Full-stack auth/ authorization + password reset + admin user mgmt | diegolsarmond | sprint-1 (inferido) | merged (main) | <code>backend/accounts/*</code> ,<br><code>backend/config/*</code> ,<br><code>frontend/src/services/auth/*</code> (principais) | Implementa autenticação e gestão de usuários no backend e base de integração no frontend (conforme mensagem do commit). |
| Commit<br><code>738749a</code> | Inicialização do frontend (estrutura + auth + user mgmt + UI)     | diegolsarmond | sprint-1 (inferido) | merged (main) | <code>frontend/*</code> (estrutura)                                                                                            | Base do app frontend com rotas protegidas, autenticação e páginas iniciais.                                             |

| ID/issue/<br>PR             | Título                                                          | Autor         | Labels                 | Status           | Arquivos alterados                                                                                      | Resumo técnico                                                                                                                                                    |
|-----------------------------|-----------------------------------------------------------------|---------------|------------------------|------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Commit<br>3da3dc3           | Core API<br>client + auth<br>+ user<br>services +<br>env config | diegolsarmond | sprint-1<br>(inferido) | merged<br>(main) | frontend/src/<br>services/api/<br>*, frontend/<br>src/services/<br>auth/*,<br>frontend/src/<br>config/* | Consolida um<br>client HTTP e<br>serviços para<br>auth/usuário com<br>configuração por<br>ambiente.                                                               |
| Código<br>(sem<br>issue/PR) | Reset de<br>senha no<br>frontend                                | —             | sprint-1<br>(OS)       | pendente         | frontend/src/<br>services/auth/<br>authService.ts<br>e página de reset                                  | O README do<br>frontend declara<br>suporte a “Forgot/<br>Reset password”,<br>mas existe função<br>resetPassword<br>não<br>implementada e<br>página de Reset.<br>2 |

## Sprint 2 — Captura de evidências, offline e sincronização

| ID/issue/<br>PR             | Título                                                                                    | Autor         | Labels                 | Status           | Arquivos alterados                                                                                                            | Resumo técnico                                                                                                    |
|-----------------------------|-------------------------------------------------------------------------------------------|---------------|------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Commit<br>9ffbbbc           | APIs de<br>assessment +<br>evidences +<br>integração do<br>workflow de<br>inspeção e sync | diegolsarmond | sprint-2<br>(inferido) | merged<br>(main) | backend/<br>assessments/*,<br>frontend/src/<br>features/<br>inspection/*,<br>frontend/src/<br>services/sync/*<br>(principais) | Implementa fl<br>“criar inspeção<br>upload → tran<br>estados →<br>sincronizar” e<br>com fila de syn<br>frontend.  |
| Código<br>(sem<br>issue/PR) | Worker de<br>sincronização com<br>backoff, retry e<br>transições<br>CAPTURED/<br>SYNCED   | —             | sprint-2<br>(inferido) | merged<br>(main) | frontend/src/<br>services/sync/<br>syncWorker.ts                                                                              | Worker proces<br>offline e cham<br>endpoints de<br>usando apiC<br>incluindo uplo<br>multi-part e tr<br>de status. |
| Código<br>(sem<br>issue/PR) | Captura de fotos<br>(limite 10, resize,<br>timestamp)                                     | —             | sprint-2<br>(OS)       | merged<br>(main) | frontend/src/<br>features/<br>inspection/<br>CameraCapture.tsx                                                                | Implementa c<br>via input “cam<br>limita 10 fotos<br>registra timest<br>ISO; faz resize<br>resolução mín          |

| ID/issue/<br>PR             | Título                                                                          | Autor | Labels             | Status  | Arquivos alterados                                                                          | Resumo técnico                                                                                                                    |
|-----------------------------|---------------------------------------------------------------------------------|-------|--------------------|---------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Código<br>(sem<br>issue/PR) | Persistir<br>"assessment_type/<br>environment_type"<br>no create da<br>inspeção | —     | sprint-2/3<br>(OS) | parcial | frontend/src/<br>services/sync/<br>syncWorker.ts +<br>backend/<br>assessments/<br>models.py | O modelo per<br>assessment.<br>environment<br>mas o create v<br>não os envia; i<br>relatórios e<br>classificação p<br>ambiente. 3 |

### Sprint 3 — Ciclo de vida, fila de IA e relatórios

| ID/issue/<br>PR              | Título                                                                             | Autor         | Labels              | Status           | Arquivos alterados                                                                          | Resumo técnico                                                                                 |
|------------------------------|------------------------------------------------------------------------------------|---------------|---------------------|------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Commit<br>7d652b9<br>(BE-01) | Persistir tipo de<br>avaliação e<br>ambiente na<br>RiskAssessment                  | diegolsarmond | BE-01               | merged<br>(main) | backend/<br>assessments/*,<br>backend/<br>configurations/<br>* (provável)                   | Evolui o modelo para<br>base para configuraçã                                                  |
| Commit<br>c39219c<br>(BE-02) | Expor fila de<br>processamento<br>admin via /<br>api/admin/<br>processing-<br>jobs | diegolsarmond | BE-02               | merged<br>(main) | backend/<br>assessments/<br>admin_views.py<br>etc.                                          | Endpoint administrati<br>jobs/processamento.                                                   |
| Commit<br>66bc359<br>(BE-03) | Implementar<br>geração de<br>relatório PDF                                         | diegolsarmond | BE-03               | merged<br>(main) | backend/<br>reports/*                                                                       | Existe geração de PDF<br>trechos "estáticos"/stu<br>dados de IA. Também<br>por LGPD no mesmo o |
| Código<br>(sem<br>issue/PR)  | Prefixo /api/<br>* e estrutura<br>de rotas                                         | —             | sprint-3<br>(infra) | merged<br>(main) | backend/config/<br>urls.py                                                                  | Backend centraliza ro<br>api/users, /api/a<br>admin.                                           |
| Código<br>(sem<br>issue/PR)  | Página de<br>relatórios no<br>frontend<br>(token/<br>localStorage)                 | —             | sprint-3            | merged<br>(main) | frontend/src/<br>features/<br>reports/pages/<br>ReportsPage.tsx<br>(adicionado em<br>BE-05) | Implementa listagem<br>localStorage.get<br>e concatena URL de m<br>padrão SecureStor           |

## Sprint 4 — LGPD, anonimização, IA real e integração com Olímpia

| ID/issue/PR                  | Título                                                                    | Autor         | Labels        | Status        | Arquivos alterados                                                             |
|------------------------------|---------------------------------------------------------------------------|---------------|---------------|---------------|--------------------------------------------------------------------------------|
| Commit<br>18c2fa4<br>(BE-05) | LGPD/GDPR: base legal + anonimização antes de armazenar/servir evidências | diegolsarmond | BE-05         | merged (main) | backend/assessments/*, backend/requirements.txt, backend/Dockerfile, docs LGPD |
| Documento interno            | IA pipeline: cliente real não implementado (depende do contrato)          | —             | sprint-4      | pendente      | AI_PIPELINE_INSTRUCTIONS.mo                                                    |
| Código (sem issue/PR)        | Integração com Olímpia /v2/ segurança-por-imagem/infer                    | —             | sprint-4      | pendente      | backend/assessments/ai_client.py, backend/assessments/tasks.py                 |
| Código (sem issue/PR)        | “Placas” (anonimização)                                                   | —             | sprint-4 (OS) | pendente      | backend/assessments/anonymization.py                                           |

## Sprint 5 — Homologação, hardening e entrega final

| ID/<br>issue/<br>PR | Título                                           | Autor | Labels           | Status                                          | Arquivos<br>alterados | Resumo<br>técnico                                                                                                                                                                                            | Classificação   | Link<br>GitHub |
|---------------------|--------------------------------------------------|-------|------------------|-------------------------------------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|----------------|
| —                   | Homologação/<br>monitoramento/<br>ajustes finais | —     | sprint-5<br>(OS) | não<br>encontrado<br>como<br>artefato<br>GitHub | —                     | Não há tarefas explicitamente rastreadas como "Sprint 5" no repositório; o que existe são bases e parte do hardening (LGPD), mas faltam integração IA real + padronização de docs + testes de ponta a ponta. | <b>Pendente</b> | —              |

## Checklists e plano de ação para tarefas pendentes

Abaixo estão as pendências mais relevantes (**prioridade técnica sugerida**), com checklist acionável, comandos e critérios mensuráveis. Cada item inclui uma proposta de esforço (baixa/média/alta) e impacto (baixo/médio/alto).

### Integração real com Olímpia no endpoint `/v2/segurança-por-imagem/infer`

**Prioridade:** P0 (bloqueia valor do produto)

**Esforço:** alto (depende de contrato do endpoint e auth)

**Impacto:** alto (substitui mock; libera IA real)

**Evidência técnica da pendência:** doc interno diz que cliente real depende de contrato <sup>4</sup> e `get_ai_client()` usa mock. <sup>5</sup>

**Checklist (passo a passo)**

- 1) Criar branch e preparar ambiente: - `git checkout main && git pull` - `git checkout -b feat/olimpia-ai-client`
- 2) Definir contrato do Olímpia (manual no Swagger): - Confirmar **URL base** (host do serviço), método HTTP, schema do body, limites de tamanho, e **autenticação**. - Confirmar se o endpoint recebe: - `multipart/form-data` com arquivo(s), ou - JSON com `base64`, ou - URL pública das imagens.
- 3) Implementar cliente real no backend: - Arquivo principal: `backend/assessments/ai_client.py` (substituir `NotImplementedError` /mock por implementação real). <sup>5</sup>
  - Criar classe (exemplo): `OlimpiaImageSafetyClient`.
  - Implementar: - autenticação (token cacheado em memória) **OU** header com chave, conforme docs; - chamada HTTP para `/v2/segurança-por-imagem/infer`; - parse da resposta → `findings` coerentes (`description`, `severity`, `location`) para alimentar `RiskFinding`.
- 4) Tornar o pipeline de processamento

compatível: - Ajustar `backend/assessments/tasks.py` para enviar **conteúdo de imagem** (bytes) ou referências válidas conforme contrato. <sup>6</sup>

- Se o Olímpia **não** acessar URLs internas, trocar de "evidence\_urls" para "evidence\_files" (abrindo `evidence.file` e enviando bytes). 5) Definir variáveis de ambiente e settings: - Atualizar `backend/config/settings/base.py` e `backend/.env.example` com chaves claras (ex.: `OLIMPIA_BASE_URL`, `OLIMPIA_CLIENT_ID`, `OLIMPIA_CLIENT_SECRET`).

6) Testes: - Unit tests com mock HTTP (ex.: `responses / requests-mock`) para: - 200 OK → cria findings + status `ai_reviewed`; - 4xx (contrato inválido) → `error_ai` sem retry; - 5xx/timeouts → retry Celery e depois `error_ai`. - Comandos (exemplos): - `cd infra && docker-compose up -d --build` - `docker-compose exec backend python manage.py test assessments.tests -v 2` 7) Critérios de aceitação (mensuráveis) - Ao sincronizar uma inspeção (`/api/assessments/{id}/sync/`), o backend: - cria `AIInferenceResult` com `status=succeeded` em sucesso, - cria pelo menos 1 `RiskFinding` com severidade não vazia, - transiciona `RiskAssessment.status` para `ai_reviewed`.

- Em falha controlada, `RiskAssessment.status=error_ai` e `AIInferenceResult.status=failed`, com `error_message` preenchida.

---

## Corrigir LGPD na API de evidências (serializer e consistência dos campos)

**Prioridade:** P0/P1 (compliance + consistência API)

**Esforço:** médio

**Impacto:** alto (expor corretamente status LGPD; evitar falso "not\_applicable")

**Evidência:** commit BE-05 introduz LGPD e anonimização, mas o serializer de evidências retorna `privacy_status` "placeholder" e não expõe campos LGPD; além disso o próprio commit carrega sinais de dificuldades de migração/logs. <sup>3</sup>

**Checklist** 1) Branch: - `git checkout -b fix/lgpd-evidence-serializer` 2) Ajustar serializer: - Arquivo: `backend/assessments/serializers.py` - Alterar `EvidenceSerializer` para incluir pelo menos: - `is_anonymized`, `anonymization_status`, `anonymized_at`, `privacy_status`. - Alterar `get_privacy_status()` para refletir o objeto real: - `is_anonymized:` `obj.is_anonymized` - `anonymization_status:` `obj.anonymization_status` - `anonymized_at:` `obj.anonymized_at` - `has_original_hash:` `bool(obj.original_file_hash)` - **Importante:** decidir se `original_file_hash` deve ser exposto (normalmente: não; só `has_original_hash`). 3) Ajustar testes: - Arquivo: `backend/assessments/tests/test_privacy_lgpd.py` (criado/alterado no contexto do BE-05). <sup>3</sup> 4) Rodar migrações e testes: - `cd infra && docker-compose up -d --build` - `docker-compose exec backend python manage.py migrate` - `docker-compose exec backend python manage.py test assessments.tests.test_privacy_lgpd -v 2` 5) Critérios de aceitação - `GET /api/assessments/{id}/` deve retornar `evidences[]` com: - `privacy_status.is_anonymized=true` - `privacy_status.anonymization_status` in `{"completed", "pending", "failed", "skipped", "processing"}` - `privacy_status.anonymized_at != null` quando completed. - Sem valores "hardcoded" de `not_applicable` para evidência imagem.

---

## Implementar anonimização de placas (ou estratégia equivalente)

**Prioridade:** P1 (LGPD/privacidade, dependendo do requisito de placas)

**Esforço:** alto (detecção robusta de placas é não-trivial)

**Impacto:** alto (compliance e risco operacional)

**Evidência:** TODO explícito para placas dentro do pacote de LGPD (BE-05). 3

**Checklist recomendado (estratégia pragmática)** 1) Branch: - `git checkout -b feat/plate-anonymization` 2) Definir abordagem: - Opção A: modelo detector (YOLO/ONNX) embarcado (mais preciso, maior complexidade de dependências). - Opção B: OCR + heurística (mais simples, mas pode falhar em cenários reais). - Opção C: delegar anonimização/detecção ao próprio serviço Olímpia (se ele oferecer "safe image preprocessing"). 3) Implementar em `backend/assessments/anonymization.py`: - Criar `detect_plates()` e aplicar blur/pixelate/blackout nas regiões. 4) Atualizar logs/auditoria: - Preencher `plates_detected` e `plates_anonymized`. 5) Testes: - Adicionar ao menos 1 teste com imagem de fixture (mesmo que sintética) para garantir execução do pipeline e atualização de contadores. 6) Critérios de aceitação - Quando `ANONYMIZATION_BLOCK_PLATES=true`, upload deve: - falhar com erro claro **ou** marcar evidência como `failed` e não servir a imagem original. - Quando habilitado, contadores em `EvidenceAnonymizationLog` refletem o processamento.

---

## Completar reset de senha no frontend

**Prioridade:** P1 (UX essencial; coerência com Sprint 1)

**Esforço:** médio

**Impacto:** médio/alto (acesso e suporte)

**Evidência:** frontend declara suporte, mas reset não está implementado no serviço. 2

**Checklist** 1) Branch: - `git checkout -b fix/frontend-reset-password` 2) Definir contrato com backend: - Rotas estão sob `/api/auth/` no backend. - Conferir payload esperado por `password-reset-confirm` (uid/token/new\_password). 3) Implementar `resetPassword` em: - `frontend/src/services/auth/authService.ts` 4) Ajustar página: - `frontend/src/features/auth/pages/ResetPasswordPage.tsx` para carregar `uid` e `token` (via params de rota ou querystring). 5) Testes: - `cd frontend && npm test` (se configurado) ou ao menos `npm run build`. 6) Critérios de aceitação - Usuário consegue: - solicitar reset, - abrir link, - submeter nova senha, - fazer login com a nova senha sem erro.

---

## Ajustar página de Relatórios (token + API client + URL)

**Prioridade:** P2 (admin/operacional)

**Esforço:** médio

**Impacto:** médio

**Evidência:** BE-05 adiciona `ReportsPage`, mas ela usa `localStorage` e concatena `env.API_URL` + `api/admin/...` de forma frágil, divergindo do padrão `apiClient` e `SecureStorage`.

**Checklist** 1) Branch: - `git checkout -b fix/reports-auth-client` 2) Padronizar client: - Substituir `fetch()` por `apiClient`. - Buscar token via mesmo mecanismo do app (`SecureStorage`). 3) Hardening: - Garantir que o `baseUrl` do `apiClient` seja consistente com `.env.example` (ideal: incluir `/api`). 2 4) Critérios de aceitação - Página `/reports` carrega relatórios quando autenticado e mostra "Acesso negado" quando não-admin, sem redirecionamentos inconsistentes.

---



## Prompts para o LLM do Olímpia e guia de chamada da API

**Nota de segurança (importante):** evite colar `clientSecret` em chats/logs. Nos exemplos abaixo ele aparece como variável (`$OLIMPIA_CLIENT_SECRET` / `{{CLIENT_SECRET}}`). O `clientId` informado (de baixa sensibilidade) pode ser referenciado diretamente.

### Guia de chamada da API (modelo genérico para validar no Swagger)

#### Endpoint informado:

- `POST /v2/segurança-por-imagem/infer`

**Headers (modelo base, a validar):** - `Content-Type: application/json` ou `multipart/form-data` - `Authorization: Bearer <token>` (se OAuth2/JWT) ou - `x-client-id: dein_hammer` + `x-client-secret: <segredo>` (se autenticação direta por headers) ou - `x-api-key: <chave>` (se API key)

#### Exemplo A — OAuth2 “client\_credentials” (assumido; confirme URL/token no Olímpia)

```
export OLIMPIA_CLIENT_ID="dein_hammer"
export OLIMPIA_CLIENT_SECRET="<COLE_AQUI_DE_FORMA_SEGURA>"
export OLIMPIA_AUTH_URL="<URL_AUTH_DO_OLIMPIA>"
export OLIMPIA_API_BASE_URL="<URL_BASE_API_DO_OLIMPIA>"

# 1) Obter token
TOKEN=$(curl -s -X POST "$OLIMPIA_AUTH_URL/oauth/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "grant_type=client_credentials" \
  --data-urlencode "client_id=$OLIMPIA_CLIENT_ID" \
  --data-urlencode "client_secret=$OLIMPIA_CLIENT_SECRET" | jq -
r .access_token)

# 2) Chamar infer
curl -X POST "$OLIMPIA_API_BASE_URL/v2/segurança-por-imagem/infer" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d @payload.json
```

#### Exemplo B — JSON com imagem em base64 (assumido; valide o schema)

```
{
  "imagens": [
    {
      "nome": "evidence_01.jpg",
      "conteudoBase64": "<BASE64>",
      "timestamp": "2026-03-16T12:34:56Z"
    }
  ],
}
```

```

"contexto": {
  "titulo": "Inspeção WorkSafety",
  "descricao": "Inspeção no canteiro",
  "tipoAmbiente": "construcao",
  "tipoAvaliacao": "epi"
}
}

```

### Exemplo C — multipart/form-data (assumido)

```

curl -X POST "$OLIMPIA_API_BASE_URL/v2/seguranca-por-imagem/infer" \
  -H "Authorization: Bearer $TOKEN" \
  -F "image=@/path/evidence_01.jpg" \
  -F "metadata={\"timestamp\": \"2026-03-16T12:34:56Z\"};type=application/
  json"

```

## Prompt pronto para o LLM — Implementar cliente Olímpia no backend (P0)

### Prompt (pt-BR)

Você é um engenheiro de software sênior. Preciso integrar o backend Django do projeto WorkSafety com a API do Olímpia no endpoint `POST /v2/seguranca-por-imagem/infer` (segurança por imagem).

#### Contexto do projeto:

- O pipeline de IA roda em background via Celery e é disparado quando a avaliação muda para `SYNCED`.
- Hoje o backend usa um cliente **mock** e o doc interno diz que o cliente real depende do contrato.
- Arquivos relevantes (repo `diegolsarmond/WorkSafety`):
  - `backend/assessments/ai_client.py` (hoje retorna Mock; preciso do cliente real)
  - `backend/assessments/tasks.py` (task `process_assessment` chama `get_ai_client().analyze_assessment(...)`)
  - `backend/config/settings/base.py` (config de ambiente; hoje existe `AI_SERVICE_BASE_URL` / `AI_SERVICE_API_KEY`)

#### Objetivo:

- Criar `OlimpiaImageSafetyClient` que chama `/v2/seguranca-por-imagem/infer` e transforma a resposta em uma lista de findings compatível com o modelo (`description`, `severity`, `location`).
- Substituir o uso do Mock por esse cliente real via `get_ai_client()`.
- Ajustar o pipeline para enviar arquivos de imagem corretamente (bytes/multipart ou base64) conforme a especificação do Olímpia (consulte a documentação do portal do Olímpia).

#### Autenticação:

- Use `clientId = dein_hammer`.

- Use `clientSecret` a partir de variável de ambiente (**não imprimir em logs**).
- Descobrir no Swagger do Olímpia se é OAuth2 `client_credentials` ou outro método de auth; implementar de acordo.

#### **Critérios de aceitação (mensuráveis):**

1. Em caso de sucesso, `AIInferenceResult.status = succeeded`, `RiskAssessment.status` transiciona para `ai_reviewed`, e são criados `RiskFindings`.
2. Em falha (4xx/5xx/timeouts), `AIInferenceResult.status = failed` (ou retry controlado), `RiskAssessment.status = error_ai`, e `error_message` fica preenchido.
3. Adicionar testes unitários com mock de HTTP validando headers e parse do response.

#### **Entrega esperada:**

- Patch de código (snippets) indicando exatamente quais arquivos alterar e como.
- Sugestão de variáveis de ambiente (`OLIMPIA_BASE_URL`, `OLIMPIA_CLIENT_ID`, `OLIMPIA_CLIENT_SECRET`, timeouts, etc).
- Exemplos de payload para o endpoint `/v2/seguranca-por-imagem/infer` conforme docs.

## **Prompt pronto para o LLM — Corrigir serializer LGPD de evidências (P0/P1)**

### **Prompt (pt-BR)**

Preciso corrigir a serialização LGPD de evidências no backend Django do WorkSafety.

**Problema atual:** `backend/assessments/serializers.py` possui `EvidenceSerializer.get_privacy_status()` retornando valores "placeholder" (`not_applicable`, `False`), e o serializer não expõe diretamente campos LGPD da evidência. Isso conflita com a implementação LGPD do commit "BE-05" e quebra consistência da API.

#### **Arquivos relevantes:**

- `backend/assessments/serializers.py`
- `backend/assessments/models.py` (Evidence possui campos LGPD)
- `backend/assessments/tests/test_privacy_lgpd.py` (ajustar conforme necessário)

#### **Objetivo:**

- Ajustar `EvidenceSerializer` para incluir `is_anonymized`, `anonymization_status`, `anonymized_at` e um `privacy_status` consistente baseado no objeto.
- Não expor `original_file_hash` diretamente (apenas `has_original_hash`).
- Ajustar/garantir testes.

#### **Critérios de aceitação:**

- `GET /api/assessments/{id}/` retorna evidências com `privacy_status` real (sem hardcode).
- Testes LGPD passam.

Gere o patch de código.

---

## Prompt pronto para o LLM — Implementar anonimização de placas (P1)

### Prompt (pt-BR)

Preciso implementar anonimização de placas de veículos no serviço de anonimização do WorkSafety.

**Contexto:** Já existe anonimização de rostos via OpenCV em `backend/assessments/anonymization.py`, mas `plates` está TODO (retorna 0). O requisito de LGPD da OS exige anonimizar faces e placas antes de armazenar/servir evidências.

#### Arquivos relevantes:

- `backend/assessments/anonymization.py`
- `backend/assessments/tasks.py` (task `anonymize_evidence_task`)
- `backend/assessments/models.py` (`EvidenceAnonymizationLog` tem campos `plates_detected` / `plates_anonymized`)

#### Objetivo:

- Implementar detecção e anonimização de placas (blur/pixelate/blackout) com uma solução pragmática (pode ser YOLO/ONNX, OCR + heurística, ou outro método).
- Atualizar logs e contadores `plates_detected/plates_anonymized`.
- Incluir pelo menos 1 teste de unidade/integrado com fixture.

#### Critérios de aceitação:

- Quando houver placa, a região é anonimizada e os contadores refletem.
- Se `ANONYMIZATION_BLOCK_PLATES=true`, o pipeline falha de modo controlado quando placa não puder ser anonimizada.

Entregue patch + dependências necessárias.

---

## Prompt pronto para o LLM — Finalizar reset de senha no frontend (P1)

### Prompt (pt-BR)

Preciso finalizar o reset de senha no frontend do WorkSafety.

**Contexto:** O README do frontend afirma suporte a “Forgot/Reset password”, mas no serviço `frontend/src/services/auth/authService.ts` a função `resetPassword()` não está implementada, e a página `ResetPasswordPage.tsx` existe porém não conclui o fluxo.

O backend expõe rotas sob `/api/auth/` (ex.: `/api/auth/password-reset/` e `/api/auth/password-reset-confirm/` — confirmar no código).

#### Objetivo:

- Implementar `resetPassword()` chamando o endpoint correto com payload correto

(token + uid e nova senha, ou conforme contrato do backend).

- Ajustar a página para ler corretamente `uid` e `token` do link.
- Garantir UX de sucesso/erro.

#### Critérios de aceitação:

- Fluxo completo: solicitar reset → abrir link → trocar senha → login com nova senha.

Forneça patch de código (quais arquivos alterar e como).

## Diagrama de dependências e fluxo recomendado

```
flowchart TD
    A[Frontend: captura fotos + fila offline] --> B[Backend: /api/assessments + evidences]
    B --> C[Anonimização LGPD]
    C --> D[Status SYNCED -> Celery process_assessment]
    D --> E[Cliente IA REAL (Olímpia /v2/segurança-por-imagem/infer)]
    E --> F[Persistir AIInferenceResult + RiskFindings]
    F --> G[Validação humana + finalização]
    G --> H[Relatório PDF + download]

    subgraph Pendências críticas
        E2[Implementar cliente Olímpia real]
        C2[Placas + serializer LGPD]
    end

    E2 --> E
    C2 --> C
```

**Sugestão de priorização (curta e objetiva):** - **P0:** Integrar Olímpia (substituir mock) + corrigir serializer LGPD (evitar “falso compliance”).

- **P1:** Reset de senha no frontend + placas (se requisito mandatário) + robustez do PDF.

- **P2:** Alinhamento de documentação/rotas e padronização de client/token no frontend (ReportsPage).

#### 1 README.md

<https://github.com/diegolsarmond/WorkSafety/blob/57d86d7cc9011914c1fb15d3fcb211a57b7bf370/README.md>

#### 2 backend/README.md

<https://github.com/diegolsarmond/WorkSafety/blob/57d86d7cc9011914c1fb15d3fcb211a57b7bf370/backend/README.md>

#### 3 BE-05 — Implementar LGPD/GDPR: base legal + anonimização antes de armazenar/servir evidências

<https://github.com/diegolsarmond/WorkSafety/commit/18c2fa4b33ed7ae6f438bfff2432e8636a2e29f9>

#### 4 frontend/src/services/api/apiClient.ts

<https://github.com/diegolsarmond/WorkSafety/blob/57d86d7cc9011914c1fb15d3fcb211a57b7bf370/frontend/src/services/api/apiClient.ts>

5 **backend/assessments/tests/test\_lifecycle.py**

[https://github.com/diegolsarmond/WorkSafety/blob/57d86d7cc9011914c1fb15d3fcb211a57b7bf370/backend/assessments/tests/test\\_lifecycle.py](https://github.com/diegolsarmond/WorkSafety/blob/57d86d7cc9011914c1fb15d3fcb211a57b7bf370/backend/assessments/tests/test_lifecycle.py)

6 **backend/assessments/tasks.py**

<https://github.com/diegolsarmond/WorkSafety/blob/57d86d7cc9011914c1fb15d3fcb211a57b7bf370/backend/assessments/tasks.py>